

ArchToolKit manual

Everything the toolkit does, page by page: what goes in, what comes out, and what it checks. To print it or save it as a PDF, use your browser's Print (Ct r l + P); the navigation is left off the printout.

1. Getting started

ArchToolKit is a set of web pages for architecture work on VMware and multi-cloud estates: reading a VMware inventory, sizing and writing a VMware Cloud Foundation (VCF) 9.1 deployment, generating Terraform and Ansible, editing infrastructure files, and planning application migrations. It runs entirely in your browser. There is no server-side component, no account, and no network call from the pages.

Running it

The pages have to be opened through a small local web server, not by double-clicking the files. Browsers block the pages' code when they are opened straight from disk.

YOU HAVE	DO THIS
Node.js 22.6 or newer, on Windows	Double-click <code>start.bat</code> in the toolkit folder. It builds, finds a free port between 8080 and 8091, starts the server and opens your browser. Options: <code>--dev</code> (rebuild on every change), <code>--port N</code> , <code>--no-open</code> .
Node.js, any system	<code>npm run build</code> then <code>npm run serve</code> , and open the address it prints. <code>npm run dev</code> does both and rebuilds as files change.
Python but no Node	The built pages are already in the folder. From <code>web\</code> , run <code>python -m http.server 8080</code> and open <code>http://localhost:8080</code> .
An internal web server	Copy the <code>web</code> folder to it as static files. Nothing else is needed.

On a work laptop without admin rights: use the Node.js *.zip* download, extract it to a folder of your own, and add that folder to your account's PATH ("Edit environment variables for your account"). Or ask IT; Node.js LTS is often in Company Portal or Software Center. There is no `npm install` step: the toolkit has no dependencies.

The pages

PAGE	WHAT IT IS FOR
VMware Inventory	Import RVTools (or the collector's JSON). Capacity, clusters, VCF 9.1 host readiness, and what blocks each VM from moving. Start here: the other pages

	work from what you import.
VCF Sizing	Size a VCF 9.1 fleet: management domain, workload domains, hosts, storage, licensing, IPs.
VCF Spec Builder	Write the VCF Installer's deployment specification (SddcSpec JSON), checked against the 9.1 schema.
Data Editor	Open and change JSON or YAML files: VCF specs, playbooks, Terraform JSON, CloudFormation, ARM, cloud IAM policies, F5, Kubernetes.
Terraform	Generate Terraform for AWS, Azure, Google Cloud, OCI and vSphere, including rehosting an imported cluster.
Ansible	Generate Ansible playbooks plus their <code>requirements.yml</code> , including inventories built from the estate.
Terraform Map	A reference: which Terraform resource does which job, per cloud and per domain.
Multi-Cloud	A four-step wizard that produces a cloud recommendation and onboarding playbook, ready for Word.
Network	Configuration for switches, routers, firewalls and load balancers, with the checks, the back-out, a playbook and a change record.
Migration	Evaluate an application — route, readiness, risk, target cloud, services, plan — and keep the portfolio, with its waves, on the same page.

2. Where your data goes

Nowhere. Files you open are read by the page in your browser and never uploaded. What the toolkit keeps, it keeps in your browser:

WHAT	WHERE	HOW LONG
The imported estate (the inventory)	Browser IndexedDB	Until you press Forget or Clear all . It survives closing the browser.
The migration portfolio (every application evaluated)	Browser IndexedDB	Until you empty it on the Portfolio tab, or press Clear all . It survives closing the browser.
Work passed between pages (sizing → spec builder, spec → editor), the chosen cloud, the Data Editor's open file	Browser session storage	Until the tab is closed.

Settings files you Save	Your Downloads folder	Yours to keep. Passwords and keys are never written into them.
--------------------------------	-----------------------	--

A VCF Installer export carries the ESX root password in clear text. The Data Editor counts secrets like this and has **Download without secrets**; the spec builder has **Redact secrets in output**. Use them before sending a file to anyone.

3. Things every page shares

The version, top left

Under the ArchToolKit name, in the form *Version 2.1.0*. It is the toolkit's own version, not the version of anything it generates. Quote it if you report a problem, and check it when you are handed a copy — a zip carried into an air-gapped environment months ago will say so here.

The licence line, bottom

One line across the foot of every page: the version, the copyright, what you may do with this version, and the reminder that everything the toolkit produces is a draft for review. It links to [Licence and disclaimer](#) for the full terms.

Clear all (top right, red)

Empties the whole toolkit in this browser: the imported estate, everything passed between pages, every page's saved state, and whatever the page is showing. Press it once to arm it (it reads *Click again to clear everything*), then again within five seconds. It clears every page, not just the one you are on, because the pages feed each other: clearing one page alone would be refilled from the estate by the next.

The estate strip

Under the heading of most pages. With nothing imported it reads *No estate loaded* and offers **Import RVTools...** (you can also drag files onto it). Once imported it shows the source, VM, host, cluster and vCenter counts, with:

- **Open**: go to the inventory page.
- **Import RVTools...**: replace the estate.
- **Forget**: delete the estate from this browser.

It accepts an RVTools `.xlsx` as RVTools wrote it, individual RVTools tabs saved as `.csv`, the collector's `.json`, or several of these at once (they are merged).

Load, Save, Clear

On the Spec Builder, Terraform and Ansible pages, a strip above the form:

- **Load...**: fill the form from a settings file saved earlier (JSON, YAML or TXT).
- **Save**: download the form as a settings file, in the format picked beside it: **JSON**, **YAML**, or **Text (field = value)**, one answer per line, easy to read in an email and safe to edit in Notepad. All three load back identically.
- **Clear**: reset this page's form to its defaults (press twice). Unlike Clear all, it touches nothing else.

Passwords and keys are never saved. A file from another page is refused with a message saying which page it came from. On Terraform and Ansible the file also carries the whole **build list**, so a stack or a site playbook is one file you keep, share or come back to; **Clear** empties the list as well as the form.

The build list

On Terraform and Ansible, above the steps. One blueprint writes one thing; a migration or a landing zone is a dozen of them, so the build list assembles them into a single project — a Terraform *stack* ([section 8](#)) or an Ansible *site playbook* ([section 9](#)). Add each piece with **Add to build**, reorder, edit or remove items, then generate the lot. A field can take a value from what the other items in the list already make, through the → box beside it.

The cloud is chosen once

The platform picked on the Multi-Cloud wizard, Terraform, Terraform Map or Ansible is shared by all four for the rest of the tab. Change it on any of them and the others follow.

Findings

Checks are reported as findings: **error** (it will fail or be refused), **warning** (it will work but probably is not what you want) and **info**. Most carry what to do about it and where the rule comes from. Figures marked Verified come from Broadcom's API reference, TechDocs or a real deployment spec; Community ones from community sources.

4. VMware Inventory

Import once; every other page then works from it.

What it reads

Every tab RVTools 4.x writes: vInfo, vCPU, vMemory, vDisk, vPartition, vNetwork, vCD, vUSB, vSnapshot, vTools, vSource, vRP, vCluster, vHost, vHBA, vNIC, vSwitch, vPort, dvSwitch, dvPort, vSC_VMK, vDatastore, vMultiPath, vLicense, vFileInfo, vHealth and vMetaData, across

any number of vCenters. The [collector](#)'s JSON adds what RVTools does not capture, such as NVMe devices and NIC speeds.

What it shows

- **Estate:** hosts, VMs, physical cores and sockets, physical and allocated memory.
- **Take it further:** buttons to size VCF, decide where it goes, and generate Terraform or Ansible from it.
- **Source:** the vCenters, their versions and collection times, and rows read per tab.
- **Consolidation:** vCPU:pCPU (warns above 4:1), memory commitment (amber above 80%, red above 100%), provisioned vs consumed storage, the thin-provisioning gap, RDMs.
- **Clusters:** per cluster hosts, VMs, cores, memory, vCPU, vRAM, storage, ratios and CPU models (mixed models flagged).
- **Moving the VMs:** every VM sorted into *Ready as they are*, *Need attention*, *Blocked* and *Blocked for a cloud*, with the reason, what to do, and example VMs.
 - Blockers: disconnected, orphaned or inaccessible VMs; physical-mode RDMs; disks shared between VMs; PCI passthrough.
 - Cautions: virtual-mode RDMs, USB, connected CD/DVD, Fault Tolerance, high latency sensitivity, snapshots, disks needing consolidation, VMware Tools not running, guest OS past support.
 - Notes: independent disks, old virtual hardware, E1000 NICs, large VMs, DRS rules, vApps, SRM protection, BIOS firmware, powered off over 90 days.
- **Health and licences:** RVTools health messages and licence use (keys shown by their last five characters only).
- **VCF 9.1 host readiness:** per host, ESX version (8.0 U3a minimum), memory (128 GiB for vSAN ESA), NVMe, NIC speed (25 GbE), NIC redundancy, cores per CPU (16) and TPM, with ✓ ✗ ! and ? (? means the source does not have that fact).
- **VCF licensing exposure:** billable cores under the 16-cores-per-CPU minimum, and the hosts paying for cores they do not have.
- **As a VCF fleet:** a first-pass sizing on default assumptions, with **Open in sizing** and **Download sizing input (JSON)**.
- **Findings:** overcommit, full datastores, ESX too old to converge, mixed CPUs, hosts in maintenance mode and more.

Download canonical inventory (JSON) saves the merged estate as `atk-inventory.json`, which can be imported again later on any machine.

5. VCF Sizing

Inputs on the left; results recalculate on every change.

From your estate

With an estate imported, a panel above the form sizes it for you. Choose:

- the **management domain**: new hosts, or converge an existing cluster (one that looks like management is suggested);
- the **target host** profile, raw vSAN capacity per host, how to split **workload domains** (per vCenter, per datacenter, or one);
- vCPU per core (default 4:1), memory to use (90%), growth (20%), storage basis (consumed or provisioned), workload storage type, whether powered-off VMs count, and which source clusters are in scope.

It shows a table per domain and cluster with the hosts needed and what set the number (CPU, memory, storage or the minimum).

The form

- **Deployment**: path (greenfield, converge to management domain, import as workload domain), profile, instances in the fleet, management hosts, topology (standard, stretched, two-node) and principal storage (vSAN ESA, vSAN OSA, NFS v3, VMFS on FC).
- **Per-host hardware**: sockets, cores, hyperthreading, RAM, raw storage, NICs.
- **Tenant workload**: vCPU, RAM and capacity to host.
- **Options**: VCF Automation and size, NSX Edge cluster, edge size and nodes, N+1, target vCPU:pCPU.

Minimum management hosts

SCENARIO	MINIMUM
Greenfield, vSAN, single site	4
Greenfield, vSAN, stretched	8
Greenfield, external storage (NFS or FC)	3
Converge, vSAN	3
Converge, external storage	2
Converge, stretched	4

Results

A verdict (*Viable*, *Review* or *Blocked*), management-plane demand per component, storage with RAID and failures-to-tolerate overhead, licensing, IP and FQDN counts per network, and findings. **Download sizing report (JSON)** saves the lot. **Continue in the spec builder** carries

the result forward; with an estate loaded the spec builder also picks up the latest sizing on its own.

The management-plane figures are Broadcom's published aggregates. The live installer's resource calculation (POST /v1/sddcs/resources-calculation) is the authority.

6. VCF Spec Builder

Writes the complete VCF 9.1 deployment specification (sddcspec) the VCF Installer takes, including the parts other builders leave out: vsphereClusterSpec, fleet services, VPC/DTGW, LACP and dual-stack.

How it fills itself in

From your sizing (*Prefilled from your sizing*) or, with only an estate imported, from the estate sized on default settings (*Prefilled from your estate*): hosts, DNS, NTP, domain, and the management, vMotion and vSAN networks. Check them, then name the components.

The form

- **Instance:** SDDC ID, instance name, domain suffix, name prefix, target version, and the **deployment scenario**, one of Broadcom's eight: new VCF fleet, new VCF instance, deferred components, converge to a new fleet, converge to a new instance, new vSphere Foundation, converge to VVF, and management services plus license server for VVF.
- **Hosts, Services** (DNS, NTP), **Networks** (management, VM management, vMotion, vSAN, host TEP).
- **Fleet-level components:** management network model (shared VLAN, dedicated VLAN, with NSX overlay, with stretched overlay), fleet network, pool name, services runtime CIDR, dual stack.
- **Storage and scale:** storage type and its details (vSAN options, NFS servers and path, FC datastore names), FTT, HA profile, appliance sizes, EVC, datacenter and cluster names.
- **Switching and NSX:** vDS profile and MTU, TEP-less (9.1.1+), LACP, VPC with distributed transit gateway.
- **Components:** certificate mode, CEIP, which of Operations, Automation, management services and Identity Broker to include (the scenario fixes some), reusing an existing vCenter and other components (brownfield), and **Redact secrets in output**.
- **ESX hosts:** hostname, root password and thumbprints per host; **Add host, Regenerate from name base**. Without thumbprints, the spec skips thumbprint validation.

Output

The status (*Valid*, *Review*, *Invalid*), the generated JSON, and findings from the 9.1 validator. Buttons: **Download SddcSpec**, **Edit as JSON** (opens it in the Data Editor) and **Copy JSON**. **Validate an existing specification** at the bottom checks a spec made anywhere else. Specs written for 9.0 are rejected, because they reference the fleet management appliance 9.1 removed.

Load / Save / Clear keep the builder's settings in a file (see [section 3](#)). A deployment spec itself cannot be loaded into the form; open it in the Data Editor instead.

This page checks every documented rule it knows. Before deploying, validate on the live installer with `POST /v1/sddcs/validations`; that check is authoritative, this one is not.

7. Data Editor

Open a JSON or YAML file, change it as a form or as text, and have it checked for its kind of file as you go.

Opening

Pick a file or paste text and press **Open pasted text**. The kind of file is detected; change it in **Kind of file** if the guess is wrong.

FAMILY	FILES IT KNOWS	WHAT IT CHECKS
VMware Cloud Foundation	VCF 9.1 deployment spec	The spec builder's full 9.1 validator.
Ansible	Playbooks and task lists; YAML inventories	One module per task, fully-qualified names, modules that exist in their collection (with "did you mean"), play keywords, inventory structure.
Terraform	.tf.json, .tfvars.json	Resource and data source types exist in the provider, secret variables marked sensitive, no credentials in provider blocks.
AWS	CloudFormation (JSON or YAML, SAM too); IAM policies	Sections and attributes, Ref/GetAtt/Sub/DependsOn/Condition targets exist; IAM grammar, condition operators, full-admin and public-access warnings.
Google Cloud	IAM policies; Deployment Manager configs	Role and member formats, public members, version 3 for conditions. Deployment Manager is flagged as retired (support ended 1 April 2026; the service is turned down after 30 June 2027).

Azure	ARM templates, parameter files, Azure Policy	Sections, resource type and API version formats, parameters()/variables() references, secure parameter types; policy conditions, operators and effects.
Oracle Cloud	IAM policies	Each statement against OCI's policy grammar; tenancy-wide admin and any-user warnings.
F5 BIG-IP	AS3 and Declarative Onboarding declarations	Classes, required properties and allowed values from F5's own schemas; every pointer names something declared.
Kubernetes	Manifests, including several documents in one file	Names, removed API versions, secrets written in the manifest, unpinned images.
Anything else	Any JSON or YAML	Readable, and secrets counted.

Two views

- **Form:** every field as a control. Fields with a fixed set of answers are dropdowns (with *Other...* for a value outside the list). Lists have add, copy, move and remove; fields can be added, renamed and removed.
- **Text:** the file as written, with line numbers. Click a finding and it jumps to its line.

A form edit writes the file again from its data, so **YAML comments are lost**. The page warns you first; edit in the Text view to keep them. A file using a tag a form cannot carry (such as Ansible's `!vault`) opens in Text only.

The side panel

- **Kind of file** and **Format** (JSON or YAML; switching converts the file).
- Checks, Changed (from the file as opened) and Secrets held.
- **Download, Download without secrets** (every clear-text secret replaced by `<REQUIRED>`), **Undo, Back to as opened, Close**.
- **Find and replace** across every value (never field names), for a new domain, a re-addressed subnet or a new site code.
- The list of changes; click one to go to it.

8. Terraform generator

1. **Step 1:** pick the **Platform** and the **Blueprint**.
2. **Step 2:** fill in the parameters. Dropdowns you can type into have *Other — type a value....* Registry modules keep their less common inputs in collapsed, searchable sections. Press **Generate Terraform (HCL)** for this one thing, or **Add to build** to put it in the build list and generate the whole stack later.
3. **Step 3:** the generated files, each with **Copy** and **Download**, and **Download all as one file**. For a registry module, *What this will create* lists each resource as Created, Not created or Depends. Save the result as `main.tf` and run `terraform init` and `terraform plan`.

Building a whole stack

One blueprint writes one thing; a migration or a landing zone is a dozen of them. The **Build list** above the steps assembles them into one project:

1. Pick a blueprint, fill it in, name it in **Module / file label**, and press **Add to build** instead of Generate.
2. Repeat for each piece. In any text field, the `-->` box beside it offers what the items already in the list create — a VPC's id, a module's outputs — so the next item references them instead of a typed-in id.
3. Name the stack and press **Generate stack**.

What comes out is a Terraform root module, ready for `terraform init`:

FILE	WHAT IT HOLDS
<code>versions.tf</code>	One terraform block, with every provider the items need merged into it.
<code>providers.tf</code>	One block per provider. Add an alias here if two items need different regions or accounts.
<code>variables.tf</code>	Every variable the items declare, plus a declaration for anything they used without declaring.
<code>01-...tf, 02-...tf</code>	One file per item, numbered in list order.
<code>outputs.tf</code>	Each item's outputs, prefixed with the item's name.
<code>README.md</code>	What is in the stack and how to apply it.

Items in the list can be reordered, edited (it loads back into the form) and removed. Two copies of the same blueprint are fine: the second one's resources are renamed after the item, declaration and references together, and the page says so. Save and Load carry the whole list, so a stack is one file you can keep, share or come back to.

The file numbers are for reading. Terraform works out the real order from the references between items.

Blueprints

PLATFORM	BUEPRINTS
AWS	EC2 + security group, S3 bucket, VPC baseline, RDS PostgreSQL; modules: VPC, EC2, security group, S3, RDS, Aurora, EKS, ALB, Lambda, KMS, DynamoDB, Auto Scaling, ECS, ACM.
Azure	Linux VM + NIC + NSG, secure storage account; modules: resource group, VNet, NSG, VM, storage, Key Vault, AKS (two), web app, naming.
Google Cloud	Compute Engine VM, GCS bucket; modules: VPC, GKE, project factory, buckets, Cloud SQL (PostgreSQL, MySQL), service accounts, Cloud NAT, Cloud Router, Cloud DNS, KMS, log export, BigQuery.
OCI	VCN baseline, compute instance, Object Storage bucket; modules: VCN, OKE, compute, bastion, DRG, operator host.
vSphere	VM from template, tag and folder an existing VM.
Linux / Windows	Remote-exec on a host (SSH or WinRM).

From your estate

- **Rehost a cluster's VMs** (each cloud): a target instance per VM, sized from its vCPU and memory with a disk per VMDK, plus `rehost-plan.csv`. The data itself moves with the cloud's replication service (AWS MGN, Azure Migrate, Migrate to VMs, OCI Cloud Migrations).
- **VCF landing zone for a cluster** (vSphere): port groups and VLANs, folders, resource pools, custom attributes and DRS rules copied from the source cluster.

Every resource type is checked against the committed provider catalog (AWS 6.65.0, AzureRM 5.6.0, Google 8.3.0, OCI 9.2.0, vSphere 2.17.1, VCF 0.18.2, fetched 20 September 2026), so a renamed resource fails here rather than at `terraform plan`. No credentials are written: providers read them from the environment (for example `VSPHERE_USER` and `VSPHERE_PASSWORD`).

9. Ansible generator

The same three steps as Terraform, and the same build list: **Add to build** collects playbooks, **Generate site playbook** writes the lot. The output is the playbook plus a `requirements.yml` listing only the collections its modules use, pinned to the current major version. Install them with `ansible-galaxy collection install -r requirements.yml`, then test with `ansible-playbook -i inventory <file> --check --diff`.

PLATFORM	PLAYBOOKS
AWS	EC2, S3, VPC + subnets, RDS, DynamoDB, SNS, SQS, KMS, CloudWatch alarm, CloudTrail.
Azure	Linux VM, Windows VM, VNet, storage, SQL, App Service, AKS, Log Analytics, Key Vault.
Google Cloud	VM, VPC + firewall, GCS, Pub/Sub, Cloud SQL, GKE, service account, Cloud Functions, Cloud Run.
OCI	VM, Object Storage, VCN, load balancer, Autonomous DB, OKE, KMS vault key.
vSphere	VM from template, power and snapshots, extra disk, folder and annotation, reconfigure.
Linux	NGINX, security updates, users and groups, cron job, SSH hardening.
Windows	AD domain join, IIS, Windows Updates, local user.

Building a whole site playbook

The same **Build list** as the Terraform page, for Ansible. Add each playbook with **Add to build**, then **Generate site playbook** writes a repository you can run in one go:

FILE	WHAT IT HOLDS
site.yml	Imports each playbook, in list order.
01-...yml, 02-...yml	One playbook per item.
group_vars/all.yml	The answers more than one playbook gave the same way, as variables every play can see.
requirements.yml	Every collection the playbooks use, merged and pinned once.
inventory/hosts.yml	A starting inventory with the groups the plays name.
README.md	What it runs and how.

Two plays cannot hand values to each other the way two Terraform resources can — a registered variable belongs to its play. What every play can see is `group_vars/all.yml`, so that is what the `→` picker offers and where shared answers go: give two playbooks the same vCenter and it becomes one `vcenter_hostname` that both reference, changed in one place. A `{{ variable }}` nothing sets is declared there empty, so the run fails clearly rather than acting on a blank. Anything that looks like a password is deliberately left out, with a note to supply it through `ansible-vault` or `-e`.

From your estate

- **Inventory of a cluster's VMs** (all, Linux or Windows): a `hosts.yml` grouped by OS, cluster and folder.
- **Before the move: snapshots and worklist**: reports the VMs the readiness checks flagged; removes snapshots only with `remove_snapshots=true`.
- **After the move: DRS rules and attributes**: recreates affinity rules and custom attributes on the target vCenter.

Module names are checked against the committed Galaxy catalog: `amazon.aws`, `community.aws`, `azure.azcollection`, `google.cloud`, `oracle.oci`, `vmware.vmware`, `vmware.vmware_rest`, `community.vmware`, `ansible.posix`, `ansible.windows`, `microsoft.ad` and `community.general`.

10. Terraform Map

A reference for AWS, Azure, Google Cloud and OCI: for each domain (identity, networking, compute, storage, databases, security, landing-zone patterns), which Terraform resources do the job and the pattern around them, with examples. Every resource name is checked against the provider catalog; one renamed since the map was written is marked, with the closest current name in the Findings card.

It answers a different question from the generator — *what should I use for this?* rather than *write it for me* — so it stays its own page, and the two link to each other: **Which resource does what** → in the generator's step 1, and **Write some of this** → on the map. Both follow the platform you picked.

11. Multi-Cloud wizard

Choose the cloud once in **Designing for** (Azure, AWS, Google Cloud, OCI), then four steps:

1. **Initiative & basics**: type (new service, change, maintenance, migration), name, architecture, traffic, latency, team strengths.
2. **Path details & data**: data pattern and sensitivity, geography, integration, source environment, 7R approach, tooling, compliance; plus questions for the initiative type chosen.
3. **Non-functional, security & migration tooling**: criticality, uptime, RTO, RPO, ops maturity, security baseline, identity, secrets, data protection, perimeter, F5 usage, security operations.
4. **Sizing & environments**: users, requests per second, data volume, ingest, retention, environments, non-prod scale, regions.

Missing answers are pointed out but never stop you. **Generate recommendation** (or **Finish**) produces eleven sections: compute, data and storage, integration, operations and governance,

security and network, controls and cyber checklist, migration focus, DR, assumptions and gaps, sizing footprint, and an implementation playbook. **Full view**, **Print** (or save as PDF) and **Save as Word**.

With an estate imported, **Answer from the estate** fills in what the inventory can answer (the migration approach, data volume, environments, regions and a description of the estate) for the whole estate or one cluster. Criticality, uptime, sensitivity and deadlines stay yours.

12. Application Migration and Portfolio

One page for both halves of the job: evaluating an application, and holding the portfolio of everything evaluated so far. They used to be two pages that only met through an exported file, so the dashboard was always out of date. Evaluating an application now adds or replaces its row straight away, and the old portfolio address redirects to the Portfolio tab.

The tabs

- **Intake:** name, owner, criticality, RTO, RPO, workload type, enterprise standard cloud, stack, OS, database, hosting, integration types, architecture pattern, vendor, identity, integration count, data size, notes; the compliance scopes (FedRAMP, CJIS, HIPAA, PCI DSS, SOX, GDPR, ITAR, CMMC and the rest); and the hard constraints — obsolete, a SaaS replacement exists, must stay on-premises, hardware-bound, mainframe-bound, data sovereignty required.
- **Ratings:** the six factors, 1 to 5. Every one reads the same way — **1 is the worst case for a move and 5 the best** — so raising any rating always raises readiness. The score updates as you move the sliders, and each factor says what 1 and 5 mean and how much of the score it carries.
- **Results:** readiness, course of action, target cloud, risk band and wave, each with the sentence that explains it; the plan; and the services that cover each capability on that cloud. *Next application* → clears the form for the next one, and *Update the portfolio*, *Download record (JSON)* and *Copy the plan* are here too.
- **Portfolio:** the compiled list of every application you have been through, the wave planning mode, the filters, the counts, and the files in and out.
- **How it works:** the framework itself — the order the verdict is reached in, the seven routes and what each wave means.

Working through a list of applications

Evaluate & add to portfolio is one action, on both the Intake and the Ratings tab: it scores the application, routes it, and puts it in the portfolio in the same press. An application needs a name first — that is what its row is called, and evaluating the same name again replaces that row rather than adding a second. From the Results tab, **Next application** → empties the form for the

next one; the portfolio keeps everything you have already been through, and says how many that is.

In the Portfolio table, the application's name is a toggle: it opens the row out into the whole record — the intake exactly as it was answered, the six ratings, the verdict with its reasons, the wave, the services for that cloud, your notes, and when it was evaluated. That is what makes the portfolio a compiled list of the work rather than eight columns.

How the route is decided

RULE, IN ORDER	ROUTE
Marked obsolete	Retire
A SaaS replacement exists, and nothing requires it to stay on-premises	Repurchase
Must stay on-premises, hardware-bound or mainframe-bound	Retain
Readiness 80–100 / 60–79 / 40–59 / 20–39 / 0–19	Refactor / Replatform / Rehost / Retain / Retire

The six factors are weighted: cloud compatibility 20, technical debt 20, refactor effort 20, compliance 15, modularity 15, vendor lock-in 10. The target cloud is the enterprise standard when one is set; otherwise regulated or sovereign data goes to Azure on a Microsoft-heavy stack and AWS otherwise, an Oracle estate to OCI, a Microsoft estate to Azure, analytics and AI to Google Cloud, and AWS when nothing else speaks up. The risk band comes from criticality, RTO, RPO, integration count and compliance scope; it changes the wave, never the route.

A correction from the page this was ported from. The old scoring inverted four of the six factors, so rating technical debt, vendor lock-in, compliance complexity or refactor effort at 5 — the best case, by the label beside it — lowered readiness. The labels were right and the arithmetic was wrong, so the inversions are gone. A portfolio carried over from the old page will score differently, and more sensibly, for those four.

The waves

Wave 0 is the landing zone — identity, network, logging, guardrails, tooling — and comes before any application moves, so nothing is placed in it. Retain is **Blocked**. Retire and Repurchase go in wave 1, because they remove footprint immediately. Otherwise low risk with readiness 55 or more is wave 1, low or medium risk with readiness 40 or more is wave 2, and the rest is wave 3. *Fast exit* also puts low- and medium-risk rehosts and replatforms in wave 1; *Modernise first* does the same for refactors at readiness 70 or more. The mode changes the table in place — nothing is recalculated on disk — and never rescues a high-risk application from its own discovery.

Files in and out

ACTION	WHAT IT DOES
Import CSV	An inventory spreadsheet becomes draft rows, evaluated with default ratings. Columns: name, owner, criticality, rtoHours, rpoHours, workloadType, enterpriseStandardCloud, primaryStack, osRuntime, database, vendor, integrationCount, dataSizeGb, identity, compliance, dataSovereigntyRequired, notes, with compliance scopes separated by ;. Only name is required; a row without one is skipped and said so.
Import JSON	A portfolio exported here, a single record, or a bare list of applications — including the files the old pages wrote. Every record is evaluated again on the way in, so an import reflects today's rules rather than a stored verdict.
Export CSV	The intake and the verdict together, including the wave under the mode currently selected. Always the whole portfolio, whatever the filters show.
Export JSON	The whole portfolio, to carry to another browser or keep with the design.
Re-evaluate all	Runs every held application through the rules again — after ratings have been revised, or after a toolkit update.
Empty the portfolio	Removes every row. The header's <i>Clear all</i> does this along with everything else in the toolkit.
Remove (on a row)	Takes one application out of the portfolio. It does not touch the form.

A row's **Open** button loads that application back into the Intake tab, which is how a draft imported from a spreadsheet gets its ratings answered. An application's name is its identity: evaluating the same name again replaces the row rather than adding a second one. The portfolio is kept in this browser (IndexedDB) and survives a reload; the *Load*, *Save* and *Clear* strip at the top of the page is for one application's intake, not the portfolio.

13. Network devices

Configuration for the things between the servers: Cisco IOS and IOS-XE, Cisco NX-OS, Arista EOS, Palo Alto PAN-OS, Fortinet FortiOS, and F5 BIG-IP as AS3 declarations. It works like the Terraform and Ansible pages — pick the platform, pick the change, fill in the parameters,

generate — with one difference: the platform here is a device operating system, not a cloud, so this page keeps its own selection and does not change what the other pages are working on.

What every change carries

A Terraform plan can be read before it runs and a playbook has --check. A router has neither: the moment a line is pasted it is live. So nothing here emits configuration on its own. Every change has four parts, and they are all in the file:

PART	WHAT IT IS FOR
Capture first	The show commands to run before you touch anything, so there is something to compare against and to restore from.
Configuration	The change itself, in the device's own syntax.
Verify	What proves it worked, run after.
Back out	The exact commands that put it back.

Everything that is not configuration is commented out in the device's own comment character, so the file can be pasted whole and only the configuration takes effect — because someone will select all and paste. Each change also declares its **impact**: none (nothing in the traffic path changes), brief (traffic can move or drop for a moment), or outage (something stops passing traffic until it is complete). The page warns on anything above none.

What it can configure

PLATFORM	CHANGES
Cisco IOS / IOS-XE	VLAN and SVI (with HSRP and a DHCP helper), access ports, trunk, LACP port-channel, static route, OSPF, BGP neighbour with prefix lists and a maximum-prefix limit, named access list, management baseline (NTP, syslog, SNMPv3, SSH), spanning-tree hardening.
Cisco NX-OS	VLAN and SVI in a VRF, vPC member port-channel, VRF, OSPF underlay with BFD, management baseline on the management VRF.
Arista EOS	VLAN and SVI with VARP, MLAG member port-channel, BGP EVPN leaf, management baseline with eAPI.
Palo Alto PAN-OS	Address objects and groups, security rule (App-ID based), source and destination NAT, layer 3 interface and zone. Emitted as set commands.
Fortinet FortiOS	Address objects and groups, firewall policy with inspection profiles, VIP to publish a server, static route.

F5 BIG-IP

HTTP/HTTPS virtual server with pool and monitor, TCP virtual server, HTTPS with an Advanced WAF policy — all as AS3 declarations, which the [Data Editor](#) can open and check against the AS3 schema.

Two ways to apply it

Every change generates both. The **configuration file** is what you paste into a session or hand to whoever has the console. The **playbook** applies the same change to many devices, and it is generated for a dry run first — `--check --diff` is the closest a network device has to a plan. The two come from one structure, so they cannot say different things. Cisco and Arista changes push through the platform's `*_config` module; PAN-OS, FortiOS and F5 use their own object modules (`panos_security_rule`, `fortios_firewall_policy`, `bigip_as3_deploy`), and PAN-OS changes include the commit.

No credentials, anywhere. Enable secrets, SNMPv3 keys, BGP session passwords and API tokens are written as `<REQUIRED>` in configuration and as vault variables in playbooks. The generated inventory says which variables each platform expects. An F5 declaration references a certificate that already exists on the device; none is created or uploaded.

Building a whole change

Add to change collects steps in order — the VLAN before the trunk that carries it, the pool before the virtual server, the interface before the rule that uses its zone. Generating the change writes:

- one numbered configuration file per step, each with its own capture, verify and back-out;
- `full-<platform>.cfg` — the **complete configuration** for that device: every step *merged*, not stacked. Two steps that touch the same interface become one interface block carrying everything they set, repeated lines are dropped, and the whole thing is written in the order the platform's own running configuration uses (system, management, VLANs, interfaces, spanning tree, routing, policy). It is what you paste into a device you are building, rather than one you are changing. For F5 it is one AS3 declaration per tenant, which is the only correct way to combine them: AS3 replaces a tenant wholesale, so two declarations for one tenant would delete each other. For PAN-OS it ends with the commit;
- `apply.yml`, the same steps as one playbook, plus `requirements.yml` and an `inventory/hosts.yml` with the connection variables;
- `change-record.md` — the pre-checks, every step with its verification, the back-out in **reverse order**, and a sign-off table;
- `README.md`, which says how to apply it both ways.

Two platforms in one change stay in separate files, because that is two sessions on two devices. The back-out reverses: the last thing built is the first thing removed.

Per-step files and the full configuration are for different jobs. The step files carry the checks and the back-out and are what a change window needs. The full configuration carries neither — it is the destination, not the journey — and is what a build needs. Both are generated every time; use the one that matches what you are doing. When a full configuration is missing something a build should have (a hostname, NTP, spanning tree), the page says so rather than pretending it is complete.

The vendor Ansible collections (`cisco.ios`, `paloaltonetworks.panos`, `fortinet.fortios`, `f5networks.f5_modules` and the rest) were added in a build with no access to Ansible Galaxy, so their versions are not pinned and their module names are not yet catalog-checked. Run `npm run ansible:update` on a machine with internet access and both are fixed. The page says so in its findings until you do.

14. The inventory collector

`tools\collector\Export-AtkInventory.ps1` is a read-only PowerShell script that collects more than RVTools does, because VCF readiness depends on hardware facts RVTools lacks: NIC link speeds, NVMe presence, VMkernel adapters, boot device, TPM, and historical CPU and memory use.

```
Install-Module VMware.PowerCLI -Scope CurrentUser
.\Export-AtkInventory.ps1 -Server vcenter.corp.local
.\Export-AtkInventory.ps1 -Server vcenter.corp.local -StatDays 0 -Outp
```

Options: `-StatDays` (days of history, default 30; 0 is much faster), `-IncludeVmDetail` (per-VM disks and NICs), `-Credential`. It writes one JSON file and sends nothing anywhere. Import it on the inventory page, alone or together with the RVTools workbook.

15. Keeping catalogs current

The lists the toolkit checks against are stored in the toolkit so it works offline. Refreshing them is the only thing that uses the network, and only to read. Each finding says how old its catalog is and warns after 90 days.

COMMAND	REFRESHES	NEEDS
<code>update-catalog.bat</code>	Terraform and Ansible catalogs together, then offers to verify and rebuild	<code>registry.terraform.io</code> , <code>galaxy.ansible.com</code>

npm run catalog:update	Terraform provider resource names	registry.terraform.io
npm run modules:update	Terraform registry modules and their inputs	git, github.com
npm run ansible:update	Ansible module catalog	galaxy.ansible.com
npm run sizes:update	Cloud machine sizes used for rehosting	github.com for AWS (optional)
npm run editor:update	F5 AS3 and DO schemas for the Data Editor	git, github.com
npm run verify:schemas	Checks generated Terraform arguments against the provider docs	registry.terraform.io

A fetch that fails leaves that catalog as it was. After refreshing, run `npm run build` (and `npm test`).

16. Common workflows

From an RVTools export to a VCF deployment spec

1. VMware Inventory: import the RVTools `.xlsx` (and the collector JSON if you have it). Read the readiness and blockers.
2. VCF Sizing: in *From your estate*, choose the management domain, host profile and domain split. Check the verdict.
3. Continue in the spec builder: check the prefilled hosts and networks, choose the scenario, name the components, add host passwords. **Save** the settings.
4. Download the SddcSpec, then validate it on the live installer.

Change a deployed VCF spec

1. Data Editor: open the installer export (`vcf-deployment-spec-....json`).
2. Edit fields in the form, or use Find and replace for a new subnet or domain. Watch the checks.
3. **Download**, or **Download without secrets** to share it.

Rehost a cluster to a cloud

1. Import the estate; on the inventory page, fix or note the blocked VMs.

2. Multi-Cloud: **Answer from the estate** for the cluster, complete the rest, and save the recommendation as Word.
3. Terraform: *Rehost a cluster's VMs* for the chosen cloud; Ansible: the inventory and before/after playbooks.

Build a landing zone or a migration as one project

1. Terraform: add each piece to the build list — network first, then what sits on it. Use the → box to point each item at what the earlier ones create rather than typing ids.
2. Name the stack, **Generate stack**, and download the files into one folder. `terraform init, validate, plan`.
3. Ansible: build the matching site playbook the same way, for what has to be configured once the machines exist.
4. **Save** both lists. That file is the design: load it to change one value and generate again.

Plan a portfolio of applications

1. Migration → Portfolio: import the inventory CSV, which creates a draft row per application.
2. Open each row, answer its ratings and hard constraints on the Intake and Ratings tabs, and evaluate. The row updates in place.
3. Back on Portfolio, choose the wave planning mode and read the counts and the table.
4. Export the portfolio as CSV for the programme, or as JSON to carry it to another browser.

17. Troubleshooting

WHAT YOU SEE	WHY, AND WHAT TO DO
A blank page, or nothing happens on click, after double-clicking an .html file	The page was opened from disk. Use <code>start.bat</code> , <code>npm run serve</code> or <code>python -m http.server</code> (section 1).
node is not recognised	Node is not installed or not on PATH. Open a new terminal after installing; see the work-laptop tip in section 1.
A page fills itself in with an old estate	The estate stays until forgotten. Press Forget on the estate strip, or Clear all .
Full view or Print does nothing on the Multi-Cloud page	A pop-up blocker stopped the new window. Allow pop-ups for the toolkit's address.
"That file was saved from ... page, not this one"	Settings files belong to the page that saved them. Open it on that page.

A finding says a catalog is more than 90 days old	Run the update in section 15 , or carry on; the names are just less certain.
A build list item says its blueprint is gone	The list was saved on another platform. Switch the platform back, or remove that item.
A stack renamed a resource	Two items declared the same address, so the second was renamed after its item, declaration and references together. Rename the item if you want a different name.
A site playbook says a variable is not set	A playbook uses <code>{{ something }}</code> that nothing supplies. Fill it in in <code>group_vars/all.yml</code> , or pass it with <code>-e</code> .
A network change warns about its impact	That is the point: it is saying what the change does to traffic. Read the impact line, confirm the window, and have the back-out open before you start.
A network playbook says a module does not exist	The vendor collection is not installed, or not catalogued yet. Run <code>ansible-galaxy collection install -r requirements.yml</code> , and <code>npm run ansible:update</code> to catalog it.
The Data Editor won't show the form	The file uses a tag the form cannot carry (such as <code>!vault</code>), or the text does not read as JSON/YAML. The Text view shows the line at fault.

18. Known limits

- The spec builder's checks follow the documented 9.1 schema; the live VCF Installer's validation is the authority.
- A deployment spec cannot yet be loaded into the spec builder's form; edit it in the Data Editor instead.
- RVTools has no NVMe or NIC-speed data; those readiness columns show ? unless the collector's JSON is imported too.
- A CSV import cannot answer the six ratings or the hard constraints, so its rows are drafts scored at the defaults until you open each one.
- The portfolio lives in one browser on one machine. Export the JSON to move it anywhere else.
- Multi-Cloud's *Save as Word* writes a Word-readable `.doc` (HTML), not a native `.docx`.
- Editing YAML in the Data Editor's form drops comments; the Text view keeps them.
- In a Terraform stack, a resource offers only its `.id` to other items, because that is the one attribute every resource has. Module items offer their real outputs. Anything else you write yourself.

- An Ansible site playbook shares an answer only when two playbooks give it identically; anything named like a password is left out of `group_vars/all.yml` on purpose.
- Network changes are written for the platforms listed and checked for structure, not simulated. Diff every one against the running configuration before it is applied.
- The network vendor Ansible collections are unpinned and un-catalogued until `npm run ansible:update` runs on a connected machine.
- Checks confirm names, structure and references. They do not confirm that a deployment will succeed in your environment.

19. Licence and disclaimer

ArchToolKit © 2026 Theodore Gibson. **All rights reserved.** The full terms are in the `LICENSE` file at the root of the toolkit; this is what they say.

What you may do

Use this version, as it was supplied to you, for your own work — personal or inside the organisation you work for. Running it and reading it are covered. The permission applies to the version you have; a later version may come under different terms, and the author may withdraw it at any time.

What you produce with it — a sizing, a specification, a generated configuration, a migration plan — is yours to use as you see fit. The licence governs the toolkit, not your output.

What you may not do, without the author's written permission

- Copy, publish, distribute or share the toolkit, or any part of it, outside your own organisation.
- Modify, adapt, port or build anything derived from it.
- Sell, rent, sublicense or charge for it, or include it — or anything derived from it — in something you charge for.
- Reuse its source, data files, catalogs, wording or page designs in other software.
- Remove or alter the licence, the copyright notice, or the attribution shown in the application.

All rights not expressly granted are reserved by the author.

No warranty, no liability

The toolkit is provided **as is**, with no warranty of any kind — including merchantability, fitness for a purpose, accuracy and noninfringement — and **the author accepts no liability** for any claim, damage, loss or cost arising from it or from anything it produces, in contract, tort or otherwise.

You use it at your own risk.

What that means in practice, page by page

- **Sizing and specifications are estimates.** Validate them against Broadcom's current documentation and a live VCF Installer before you commit to hardware, licensing or a deployment window.
- **Generated Terraform, Ansible and declarations are unreviewed code.** Read them, run `terraform plan` or `--check --diff`, and test them somewhere that does not matter first. Applying them can create billable cloud resources, change running systems or interrupt service.
- **Migration routes, risk bands, waves and service recommendations are decision support.** They are a weighted reading of the answers you gave, and they do not replace the judgement of the people accountable for the applications.
- **Nothing here is legal, regulatory, certification, financial or licensing advice.** Compliance scopes and regulatory references are informational and may be out of date. Confirm them with your own auditors, legal counsel and the relevant authority.
- **Your data is yours.** The toolkit is offline and stores everything in your own browser (see [Where your data goes](#)). What you enter, and where you send what comes out, is your responsibility.

Trademarks

Vendor and product names — VMware, VMware Cloud Foundation, AWS, Microsoft Azure, Google Cloud, Oracle Cloud Infrastructure, F5, Splunk, Cisco, Red Hat, Ansible, Terraform, HashiCorp and the rest — are the trademarks of their owners, used here only to describe what the toolkit builds for. ArchToolKit is not affiliated with, endorsed by or sponsored by any of them.

ArchToolKit 2.3.0 © 2026 Theodore Gibson, all rights reserved Free to use as supplied; no copying, modifying or selling
No warranty, no liability Every output is a draft to review, not advice [Licence and disclaimer](#)